IMPORTANT — READ THIS ENTIRE INSTRUCTION BEFORE MAKING ANY CHANGES

Hey Cursor,

I want you to work on this project as a **senior ecommerce engineer + ecommerce product specialist**, not like a junior software engineer who blindly implements individual instructions.

You need to understand the **entire ecommerce scenario, existing architecture, product relationships, category structure, filters, breadcrumbs, and UI behavior** before making changes.

Do NOT make assumptions.
Do NOT invent product information.
Do NOT randomly change existing working logic.
Do NOT implement something just because it looks technically easy.

Think about how a real ecommerce website such as StarTech should behave for a customer.

---

1. FIRST: UNDERSTAND THE EXISTING PROJECT

Before writing or changing code:

1. Inspect the entire existing implementation.
2. Understand:
    - Product data structure
    - Category structure
    - Brand structure
    - Filter logic
    - Sub-category logic
    - PDP/product panel logic
    - Breadcrumb logic
    - Search/filter interaction
    - URL/state management
    - Existing components
    - Existing APIs/data sources
    - Existing image handling
3. Understand how I previously designed and fixed the **CPU, Motherboard, and RAM categories**.
4. Treat those implementations as the **reference behavior** for how category/filter logic should work.

Do not start coding immediately.

First understand how the existing system works and how the previous fixes were implemented.

---

2. VERY IMPORTANT — THINK LOGICALLY

I do NOT want a simple implementation where:

> User clicks something → code blindly filters an array.

This is an ecommerce product system.

You need to understand the relationship between:

Category → Subcategory → Brand → Product → Specification → Filters → PDP

Every selection should logically affect the available products and available filters.

Think like an ecommerce technology expert who understands:

- Product taxonomy
- Faceted filtering
- Category-specific attributes
- Brand-specific attributes
- Product relationships
- Ecommerce navigation
- Breadcrumb state
- Customer shopping behavior

Before changing anything, ask yourself:

> "If a real customer uses this website, what products and filters should logically be visible at this point?"

---

3. PREVIOUS PROBLEMS — DO NOT REINTRODUCE THEM

When I previously worked with you on CPU, Motherboard, and RAM categories, there were several logical problems.

You must make sure these problems do NOT happen again.

Problem 1 — Filters were not dependent on the selected brand

Example:

If the customer selects:

AMD

the available filters should update based on the products that actually belong to AMD.

For example, CPU-related filters such as:

- Series
- Generation
- Socket
- Core
- Thread
- Other applicable specifications

must reflect the products available under the selected brand/category.

The filter system should NOT continue showing irrelevant options from Intel or unrelated products.

The same principle applies to every category.

---

4. CATEGORY-SPECIFIC FILTER LOGIC

Filters must be **context-aware**.

Do NOT create one generic filter logic and apply it blindly to every category.

Different ecommerce categories have different important attributes.

For example:

CPU may have:

- Brand
- Series
- Generation
- Socket
- Core
- Thread
- Cache
- Frequency

Motherboard may have:

- Brand
- Socket

- Chipset
- Form Factor
- RAM Type
- RAM Slots
- Expansion Slots
- etc.

RAM may have:

- Brand
- RAM Type
- Capacity
- Bus Speed
- Module Type
- etc.

These are only examples.

The actual attributes MUST come from the real StarTech product data.

Do not invent attributes that are not present in the source data.

---

5. BRAND SELECTION MUST AFFECT PDP RESULTS

When a customer selects a specific brand:

Example:

Brand → Logitech

then the product panel should show ONLY products belonging to Logitech within the current category/context.

It should NOT show:

- Other brands
- Unrelated products
- Products from another category
- Products that only partially match the selection

The product list and filter options must remain logically synchronized.

---

6. BREADCRUMB LOGIC

Breadcrumbs are extremely important.

When a customer navigates through:

Category → Subcategory → Brand → Product

and then clicks a breadcrumb to go back,

the application must correctly restore the expected category state.

For example:

If the customer is inside:

Keyboard → Logitech

and clicks:

Keyboard

then the application should return to the Keyboard category correctly.

It should NOT incorrectly keep the Logitech brand selection active if the breadcrumb level indicates that the brand selection should be removed.

Similarly, if the user navigates backward through another breadcrumb level, the application must restore the correct state for that level.

Do not simply manipulate the visual breadcrumb.

The underlying product/filter state must also be correct.

---

7. STARTECH DATA REQUIREMENT

Now I want you to use the **StarTech website as the source of truth** for the requested ecommerce categories and products.

You need to crawl/research the StarTech site category by category.

IMPORTANT:

Never guess product information.

If information exists on the StarTech website, use that information.

If information does not exist:

- Do NOT invent it.
- Do NOT estimate it.

- Do NOT generate fake specifications.
- Do NOT create placeholder specifications pretending they came from StarTech.

Use only information that can actually be verified from the StarTech site.

---

8. CATEGORIES TO COLLECT

Start with the following categories.

ACCESSORIES

- Watch
- Keyboard
- Mouse
- Headphone
- Bluetooth Headphone
- Mouse Pad
- Wrist Rest
- Headphone Stand
- Speaker & Home Theater
- Bluetooth Speakers
- Soundbar
- Webcam
- Cable
- Converter
- Card Reader
- Hubs & Docks
- Microphone
- Digital Voice Recorder
- Presenter
- Memory Card
- Capture Card
- Pen Drive
- Thermal Paste
- HDD-SSD Enclosure
- Power Strip
- Bluetooth Adapter
- Monitor Light Bar
- Show All Accessories

GAMING

- Gaming Chair
- Gaming Desk
- Gaming Console

- Gaming PC
- Gaming Laptop
- Gaming Monitor
- Keyboard
- Mouse
- Headphone
- Mouse Pad
- Gamepad
- Racing Wheel
- VR
- Games
- Gaming Router
- Show All Gaming

---

9. CATEGORY-BY-CATEGORY PROCESS

For EVERY category:

1. Find the correct StarTech category page.
2. Understand the category.
3. Collect all available products/PDPs in that category.
4. Go through the products one by one.
5. Collect the actual product information.
6. Collect all available product images.
7. Collect the actual product name.
8. Collect the actual price.
9. Collect the actual brand.
10. Collect the actual category/subcategory.
11. Collect available specifications.
12. Collect available descriptions.
13. Collect available features.
14. Collect available SKU/model information if present.
15. Collect available availability/stock information if present.
16. Collect product URLs if useful for traceability.

Do not stop after collecting a few products.

The requirement is to collect **all available products that belong to the requested category**, subject to what the StarTech site actually exposes.

---

10. PRODUCT IMAGES

Images are extremely important.

For every PDP:

- Collect ALL relevant product images available on the StarTech PDP.
- Do not collect only the first/main image.
- Do not replace missing images with generated images.
- Do not use unrelated images.
- Do not guess which image belongs to which product.
- Maintain the correct relationship:

Product → Product Images

Each image must belong to the correct product.

If the site has:

- Main product image
- Gallery images
- Alternate angles
- Lifestyle images
- Product detail images
- Packaging images

collect them where available.

---

11. PRODUCT INFORMATION — NO GUESSING

This is one of the most important requirements.

Use the StarTech site as the source of truth.

For example, if StarTech says:

Brand: XYZ

use XYZ.

If StarTech says:

Price: ৳X,XXX

use that price.

If StarTech lists:

RAM: 16GB

use 16GB.

Do NOT convert, improve, simplify, reinterpret, or invent the information unless required by the existing application's UI/data structure.

The information should remain as close as possible to the source website.

---

12. PRICE ACCURACY

Price must match the StarTech website.

Do NOT:

- Estimate price
- Calculate price
- Convert currency unnecessarily
- Add fake discounts
- Create fake sale prices
- Round prices
- Generate prices from another source

If StarTech displays a specific price, use that price.

If StarTech displays:

- Regular price
- Discounted price
- Offer price
- Previous price
- Current price

preserve the correct relationship.

---

13. DO NOT USE FAKE DATA

I want to make this extremely clear:

NEVER generate fake ecommerce data.

Do not create:

- Fake products
- Fake brands
- Fake prices
- Fake specifications
- Fake images

- Fake SKUs
- Fake stock information
- Fake reviews
- Fake ratings
- Fake categories

If something cannot be verified from StarTech, leave it unavailable rather than inventing it.

---

14. PRODUCT-CATEGORY RELATIONSHIP

A product should appear in a category ONLY when it actually belongs to that category.

For example:

If we are viewing:

Gaming Mouse

do not show every mouse from the entire website unless StarTech categorizes it as a gaming mouse.

Similarly:

Bluetooth Headphone

should not automatically contain every headphone.

Use the actual StarTech category/product relationship.

Do not make assumptions based only on the product name.

---

15. CATEGORY → BRAND → PRODUCT LOGIC

The complete logic should work like this:

Example

Customer selects:

Gaming → Mouse

The product panel shows Gaming Mouse products.

Then customer selects:

Brand → Logitech

Now:

Only Logitech Gaming Mouse products should appear.

Then the available filters should update based on those Logitech Gaming Mouse products.

Then if the customer removes Logitech:

The product list should return to all Gaming Mouse products.

This relationship should work consistently across all categories.

---

16. MULTIPLE FILTERS

Filters must work together.

Example:

Category:

Gaming Mouse

Brand:

Logitech

Connection:

Wireless

Price:

৳X – ৳Y

The result should satisfy ALL selected conditions.

Not:

Brand OR Connection OR Price.

It should behave like a proper ecommerce faceted filtering system.

Also ensure that the available filter values update logically according to the current filtered dataset.

---

17. FILTERS MUST NEVER SHOW IMPOSSIBLE OPTIONS

Example:

Suppose:

Logitech Gaming Mouse products only support:

- Wireless
- Wired

Then the filter should not show an unrelated connection type simply because another product elsewhere in the database has it.

Available filter values should be based on the relevant product dataset.

This is extremely important.

---

18. DO NOT BREAK EXISTING CPU / MOTHERBOARD / RAM LOGIC

Before modifying shared filtering/category code:

Check whether CPU, Motherboard, and RAM depend on the same logic.

If you modify a shared component or function, verify that the existing CPU/Motherboard/RAM behavior still works.

Do NOT fix one category while breaking another.

After making changes, test:

- CPU
- Motherboard
- RAM
- Accessories
- Gaming

and verify that category selection, filters, products, and breadcrumbs still work.

---

19. UI BEHAVIOR

The UI should behave like a professional ecommerce website.

When the user changes:

- Category
- Subcategory
- Brand
- Filter
- Breadcrumb

the following should remain synchronized:

URL/state → Breadcrumb → Filters → Product list → Product count → Available filter values

There should never be a situation where the UI visually shows one state while the product data represents another state.

---

20. DATA STRUCTURE

Before creating a large amount of product data, inspect the existing project's data architecture.

Reuse the existing architecture where possible.

Do not create unnecessary duplicate structures.

If a better normalized structure is required, design it carefully.

For example, conceptually:

Category
→ Subcategory
→ Brand
→ Product
→ Specifications
→ Images
→ Price

But adapt this to the existing project's architecture rather than blindly implementing this structure.

---

21. PERFORMANCE

There may be a large number of products.

Do not create extremely inefficient logic such as repeatedly scanning the entire product dataset unnecessarily for every UI operation.

Think about:

- Filtering performance
- Image loading
- Rendering performance
- State management
- Memoization where appropriate
- Data normalization
- Pagination/lazy loading if already supported
- Avoiding unnecessary re-renders

But do NOT sacrifice correctness for performance.

Correct ecommerce behavior comes first.

---

22. DO NOT CHANGE THINGS UNNECESSARILY

If an existing implementation already works correctly:

LEAVE IT ALONE.

Do not refactor code just because you prefer another coding style.

Do not rename working components without a reason.

Do not replace working logic without understanding its purpose.

Every change should have a clear reason.

---

23. THINK BEFORE CODING

Before making a major implementation:

Ask yourself these questions internally:

1. What category am I currently in?
2. What products belong to this category?
3. What brands exist inside this category?
4. What attributes are actually available for these products?

5. Which filters should be visible?
6. Which filters should disappear?
7. What products should appear after a selection?
8. What should happen when the selection is removed?
9. What should the breadcrumb represent?
10. What should happen when the user clicks a breadcrumb?
11. Will this change affect CPU/Motherboard/RAM?
12. Am I using actual StarTech data or am I assuming something?

If you cannot answer these logically from the available data/code, inspect more of the project/source before implementing.

---

24. VALIDATION AFTER IMPLEMENTATION

After implementation, do not just check whether the application compiles.

Actually test the ecommerce behavior.

Test scenarios such as:

Scenario 1

Category → Brand → Products

Scenario 2

Category → Brand → Filter → Products

Scenario 3

Category → Brand → Remove Brand

Scenario 4

Category → Multiple Filters → Products

Scenario 5

Category → Brand → Breadcrumb Back

Scenario 6

Category → Subcategory → Brand → Breadcrumb Back

Scenario 7

No matching products

The UI should handle all of these logically.

---

25. FINAL PRINCIPLE

The most important instruction is:

DO NOT THINK LIKE A CODE GENERATOR.

Think like:

Senior Ecommerce Engineer + Ecommerce Product Manager + QA Engineer

You need to understand the complete customer journey.

When I give you product/category data, do not simply put that data into the UI.

Understand:

Why does this product belong here?

Why should this filter appear here?

Why should this filter disappear after another selection?

What should the customer see after selecting this brand?

What should happen when the customer goes back?

What information actually exists on StarTech?

What information is NOT available and therefore must not be invented?

---

26. IMPORTANT EXECUTION RULE

Before making large changes, first inspect the existing codebase and identify:

1. Current category architecture
2. Current product data architecture
3. Current filter implementation
4. Current breadcrumb implementation
5. CPU/Motherboard/RAM implementation
6. Existing reusable components

7. Existing issues that could affect this implementation

Then create a concise implementation approach based on what you found.

Only after understanding the existing system should you start modifying the code.

Final goal

Build the requested StarTech ecommerce category/product experience so that it behaves like a **real, production-quality ecommerce system**, with:

- Accurate StarTech product data
- Accurate StarTech images
- Accurate StarTech pricing
- Correct category relationships
- Correct brand relationships
- Logical category-specific filters
- Dynamic dependent filters
- Correct product filtering
- Correct breadcrumbs
- Correct navigation state
- No invented information
- No broken existing functionality
- No unnecessary code changes

Accuracy and logical ecommerce behavior are more important than speed.

If there is a conflict between "quickly implementing something" and "implementing it correctly according to the ecommerce logic," always choose correctness.